

Data Governance Copilot

Interview Question Bank

Topic 04 — Memory & Checkpointing

12 questions · 4 sections · MemorySaver · PostgresSaver · SqliteSaver · thread_id · Neon · Conversation History

Foundational

Core concepts

Intermediate

Design decisions

Advanced

Deep internals

Gotcha

Real bugs / traps

What Is Checkpointing & Why It Matters

Q01 Foundational

What is LangGraph checkpointing and why does a stateless HTTP API need it for a conversational AI system?

Hint: Think about the difference between a single query and a multi-turn conversation.

Key points to cover:

- LangGraph is stateless by default — each `graph.invoke()` call starts fresh with no memory of prior runs
- Checkpointing serialises `AgentState` after every node and persists it to a storage backend
- On subsequent calls with the same `thread_id`, LangGraph rehydrates the last checkpoint and continues from that state
- Without checkpointing: user asks 'what is the retention metric?' then 'why did it drop?' — second query has no context
- With checkpointing: `conversation_history` in `AgentState` accumulates across turns — LLM sees full dialogue
- FastAPI is stateless (HTTP) — checkpointing bridges the HTTP request/response model with stateful multi-turn dialogue
- Also enables HITL: `auto_ticket_node` sets `pending_action` in checkpoint; user approves later in a new HTTP request — graph resumes from checkpoint with `approved=True`

Q02 Foundational

What is `thread_id` and how does it flow through your system from the API call to the checkpointer?

Hint: Trace `thread_id` from the `POST /query` request body all the way to the LangGraph config.

Key points to cover:

- `thread_id` is a string identifier for a conversation session — client generates or persists it (e.g. UUID per chat window)
- `POST /query` request body includes `thread_id` (or API generates one if absent)
- `_run_graph` helper in `api/app.py` builds LangGraph config: `{'configurable': {'thread_id': thread_id}}`
- `graph.invoke(state, config=config)` — LangGraph uses `thread_id` as the checkpoint namespace key
- Checkpointer stores/retrieves state under that `thread_id` — all turns of the same conversation share one thread
- Different users get different `thread_ids` — their conversation histories are fully isolated
- MCP tool `approve_action` also takes `thread_id` — re-invokes the graph on the same thread to resume HITL flow
- In Teams bot: `TeamsConversation.id` is used as `thread_id` — each Teams channel thread maps to one LangGraph thread

```
# In api/app.py async def _run_graph(query, thread_id, user_id, ...): state =
initial_state(query=query, thread_id=thread_id, ...) config = {'configurable': {'thread_id':
thread_id}} result = await graph.ainvoke(state, config=config) return result
```

The Three-Tier Fallback Chain

Q03**Intermediate**

Walk through the checkpoint fallback chain in `checkpointer.py`. What triggers each level and what are the tradeoffs?

Hint: There are three levels — explain when each activates and what you lose at each step down.

Key points to cover:

- Level 1 — `PostgresSaver`: activated when `DATABASE_URL` is set and `ENVIRONMENT=production`
- `PostgresSaver` persists to Neon PostgreSQL — durable, shared across all ECS tasks, survives restarts
- Level 2 — `SqliteSaver`: activated when `DATABASE_URL` is absent or `PostgresSaver` init fails
- `SqliteSaver` writes to `data/memory.db` — durable within a single container but NOT shared across ECS tasks
- `SqliteSaver` fallback also tries `langgraph_checkpoint_sqlite` package before standard `langgraph.checkpoint.sqlite` (Bug #37 fix)
- Level 3 — `MemorySaver`: final fallback when both DB options fail — pure in-process dict
- `MemorySaver`: zero durability — restart loses all conversation history; also not shared across tasks
- Tradeoff summary: `PostgresSaver=durable+shared`; `SqliteSaver=durable+local`; `MemorySaver=fast+ephemeral`
- Dev uses `MemorySaver` (no `DATABASE_URL` set) — acceptable because dev sessions are short-lived

```
# checkpointer.py fallback chain
def get_checkpointer():
    if os.getenv('DATABASE_URL') and is_production():
        try:
            return PostgresSaver.from_conn_string(_pg_url())
        except Exception:
            pass
    # fall through
    try:
        from langgraph.checkpoint.sqlite import SqliteSaver
        return SqliteSaver.from_conn_string('data/memory.db')
    except ImportError:
        try:
            from langgraph_checkpoint_sqlite import SqliteSaver
            return SqliteSaver.from_conn_string('data/memory.db')
        except ImportError:
            pass
    return MemorySaver() # final safety net
```

Q04**Intermediate**

Bug #37 was about `langgraph.checkpoint.sqlite` not being available. What happened and how did the fallback chain fix it?

Hint: This is a real packaging / import error — explain the root cause and the fix.

Key points to cover:

- Root cause: `LangGraph` split checkpoint backends into separate optional packages across versions
 - `langgraph.checkpoint.sqlite` moved to `langgraph-checkpoint-sqlite` (separate pip package) in some versions
 - Original code: `from langgraph.checkpoint.sqlite import SqliteSaver` — raised `ImportError` in environments without the extra
 - The bug meant the code crashed at import time rather than gracefully falling back to `MemorySaver`
 - Fix: wrap both import paths in `try/except` — try `langgraph.checkpoint.sqlite` first, then `langgraph_checkpoint_sqlite`
 - If both fail: fall through to `MemorySaver` — system still works, just without SQLite persistence
 - Lesson: optional dependencies in Python require defensive imports — never assume a sub-module is available
 - Production safeguard: `requirements.txt` pins `langgraph-checkpoint-sqlite` explicitly to avoid this in prod
-

Q05**Advanced****In a production ECS Fargate deployment with 10 tasks running, what goes wrong if two tasks use SqliteSaver instead of PostgresSaver?***Hint: This is a distributed systems consistency problem.***Key points to cover:**

- Each ECS task has its own ephemeral filesystem — data/memory.db is NOT shared between tasks
- User sends message 1 → routed to Task-3 → checkpoint written to Task-3's memory.db
- User sends message 2 → routed to Task-7 (load balancer) → Task-7 has no checkpoint for this thread_id
- Result: LangGraph starts a fresh conversation — all prior context is lost mid-session
- HITL breaks completely: pending_action set in Task-3's checkpoint is invisible to Task-7 — user approval never resumes
- Solution: PostgresSaver with shared Neon DB — all tasks read/write the same checkpoint store
- ALB sticky sessions (session affinity) can partially mitigate: pin user to same task — but task restart still breaks it
- Your EKS migration uses ALB sticky sessions for MCP SSE transport — same principle applies to checkpoint continuity

Q06**Advanced****Explain the _pg_url() normalisation function in checkpointer.py. Why does the URL scheme need to be changed for PostgresSaver?***Hint: Neon uses postgresql:// but some LangGraph internals expect postgres://.***Key points to cover:**

- Neon DATABASE_URL uses scheme postgresql:// (standard psycopg2 / SQLAlchemy format)
- PostgresSaver uses asyncpg or psycopg3 under the hood — some versions require postgres:// scheme (without 'ql')
- _pg_url() reads DATABASE_URL, detects scheme, and normalises: postgresql:// → postgres:// if needed
- Also handles the reverse: if a bare postgres:// is provided, some sync drivers need postgresql:// — must handle both directions
- Neon also requires ?sslmode=require — _pg_url() ensures this query param is present even if user omits it
- pgBouncer pooled URLs (port 6432) vs direct URLs (port 5432): PostgresSaver needs direct URL for transaction-mode checkpointing
- AIRFLOW_DB_URL uses direct URL for the same reason — poolers break transaction semantics required by Airflow's metadata DB
- Bug surface: forgetting scheme normalisation causes 'could not connect' errors that look like auth failures — misleading

```
# _pg_url() normalisation
def _pg_url() -> str:
    url = os.getenv('DATABASE_URL', '') # Normalise scheme for asyncpg compatibility
    if url.startswith('postgresql://'):
        url = 'postgres://' + url[len('postgresql://'):]
    # Ensure SSL for Neon if 'sslmode' not in url
    url += ('&' if '?' in url else '?') + 'sslmode=require'
    return url
```

Conversation History & State Persistence

Q07**Intermediate**

How is conversation_history stored in AgentState and how does it grow across turns? What prevents it from growing unbounded?

Hint: Think about what happens after 50 conversation turns.

Key points to cover:

- conversation_history: List[dict] in AgentState — each entry is {'role': 'user'|'assistant', 'content': str}
- Each turn: supervisor_node appends the current query as {'role': 'user', ...}; synthesizer_node appends final_summary as {'role': 'assistant', ...}
- PostgresSaver serialises the full AgentState (including conversation_history) as JSON in the checkpoint table
- Unbounded growth risk: after 50 turns, conversation_history is a 50-entry list — passed to LLM context on every call
- LLM context window limit: Groq llama-3.3-70b has 128k token context — history must stay within budget
- Mitigation 1: sliding window — keep only last N turns (e.g. 10) in conversation_history before passing to LLM
- Mitigation 2: summarisation — periodically compress old turns into a summary string, replace with single entry
- Current implementation likely passes full history — a valid gap to acknowledge and describe the fix

Q08**Intermediate**

A user closes their browser mid-conversation with a pending HITL action. They reopen the app 2 hours later. What happens?

Hint: This tests your understanding of checkpoint durability and HITL state recovery.

Key points to cover:

- PostgresSaver has written the checkpoint with pending_action set and approved=False — fully durable in Neon
- User reopens app — client sends their thread_id with the new request (or retrieves it from localStorage)
- GET /history/{thread_id} can be called first to restore the UI — returns conversation_history from checkpoint
- Streamlit: _render_hitl_panel() checks st.session_state.pending_hitl — needs to be rehydrated from checkpoint on reconnect
- Next graph invocation with this thread_id: LangGraph reloads checkpoint, sees pending_action is set
- If user submits approval: graph resumes from auto_ticket_node with approved=True — Jira ticket created
- If user submits rejection: pending_action cleared, no ticket created — conversation continues
- Key insight: HITL durability is ONLY guaranteed with PostgresSaver — MemorySaver loses pending_action on restart

Q09**Gotcha**

What happens if you call graph.invoke() with a thread_id that already has a completed checkpoint? Does LangGraph replay the whole graph?

Hint: Think about how checkpointers handle re-entry vs continuation.

Key points to cover:

- LangGraph loads the latest checkpoint for that thread_id — it does NOT replay from the beginning
- The checkpoint captures the state AFTER the last node ran — graph continues from wherever it left off
- For a completed conversation (reached END): the checkpoint has final_summary set and _agents_ran is complete
- New invoke() with same thread_id + new query: LangGraph loads checkpoint, updates query field, restarts from START
- This is correct behaviour — same thread continues the conversation with full history in state
- Gotcha: if you forget to update the query field in state before re-invoking, the graph processes the OLD query again
- initial_state() factory in state.py always sets query from the new request — this prevents stale query replay
- Checkpoint namespace collision: two users accidentally sharing a thread_id would interleave their conversations — client must generate unique thread_ids (UUIDs)

Q10**Advanced**

Why does your project use a pooled pgBouncer URL for most connections but recommends a direct URL for PostgresSaver and Airflow?

Hint: This is about transaction semantics and connection pooler compatibility.

Key points to cover:

- pgBouncer pooled URL (port 6432): connections are multiplexed — multiple app connections share fewer real DB connections
 - pgBouncer transaction mode: connection is returned to pool after each transaction — most efficient for web apps
 - PostgresSaver problem: it uses prepared statements and advisory locks — these require session-mode pooling or direct connections
 - pgBouncer in transaction mode drops session state between transactions — prepared statements fail with 'unknown prepared statement'
 - Airflow metadata DB problem: uses SQLAlchemy with long-running transactions and connection-level settings — needs session continuity
 - Fix: PostgresSaver and Airflow both use direct URL (port 5432) — bypasses pgBouncer entirely
 - Application agents (InformationAgent via SQLAlchemy, psycopg2 for pgvector): safe to use pooled URL — simple query/response pattern
 - Neon serverless driver (neon-serverless) is an alternative: HTTP-based, avoids connection pooler issues entirely
-

Q11**Advanced**

How would you implement checkpoint TTL or pruning? After 90 days, old conversation checkpoints should be deleted.

Hint: PostgresSaver doesn't have built-in TTL — you need to design this.

Key points to cover:

- PostgresSaver stores checkpoints in a table (typically 'checkpoints') with columns: thread_id, checkpoint_id, created_at, data
 - Add created_at timestamp column if not present — needed for age-based pruning
 - Airflow nightly_refresh_dag is the natural place to run pruning: `DELETE FROM checkpoints WHERE created_at < NOW() - INTERVAL '90 days'`
 - Soft delete alternative: add a deleted_at column, filter in queries — allows recovery within grace period
 - Selective pruning: preserve checkpoints with pending_action (open HITL items) regardless of age
 - Cascading: checkpoint table may have child tables (checkpoint_blobs, checkpoint_writes in LangGraph schema) — delete must cascade
 - Monitoring: track checkpoint table size in CloudWatch — alert if it exceeds threshold (unbounded growth = bug)
 - GDPR consideration: user data deletion request → delete all checkpoints for that user's thread_ids → audit log the deletion
-

Q12

Design

An interviewer asks: 'LangGraph checkpointing stores full AgentState as JSON. That state includes agent_results with potentially large Databricks query outputs. What problems does this cause and how would you fix them?'

Hint: This is about checkpoint payload size and storage efficiency.

Key points to cover:

- AgentState includes agent_results: List[AgentResult] — each AgentResult.data dict could contain thousands of rows from Databricks SQL
- Large checkpoints: each checkpoint JSON blob could be 1-10MB for data-heavy queries — expensive to serialise/deserialise
- PostgresSaver stores checkpoint in a single JSONB column — 10MB JSONB per row is feasible but slow at scale
- Problem 1: latency — deserialising 5MB JSON on every graph node call adds 50-200ms overhead
- Problem 2: storage cost — 10k conversations × 10 turns × 5MB = 500GB of checkpoint data
- Fix 1: store only summaries in AgentState — full query results go to S3/Redis with a reference key in state
- Fix 2: compress checkpoint data — JSONB with pg_compress, or gzip before storing in bytea column
- Fix 3: separate hot and cold checkpoints — last 5 turns in Redis (fast), older turns in PostgreSQL (durable)
- Fix 4: truncate agent_results after synthesizer runs — final_summary is the distillation; raw results no longer needed in state